API Authorization & BOLA

scopes, ownership checks and where they live

Watch the 4-min video

In one line: a valid token proves who the caller is, not what they may touch; check scopes at the gateway, but check that the caller owns *this* object in the service, in the query itself, on every request.

1 Authentication is not authorization

User 42 logs in, gets a valid token, and calls `GET /orders/1001`. That order is theirs. Then they change the number to 1002 and get someone else's order: name, address, everything. The token was valid, so **authentication** worked. **Authorization** did not.

Authentication answers *who are you?* Authorization answers *what are you allowed to do, to this specific thing?* A valid token only proves identity. Every request still needs a decision: may this **caller** perform this **action** on this **resource**?

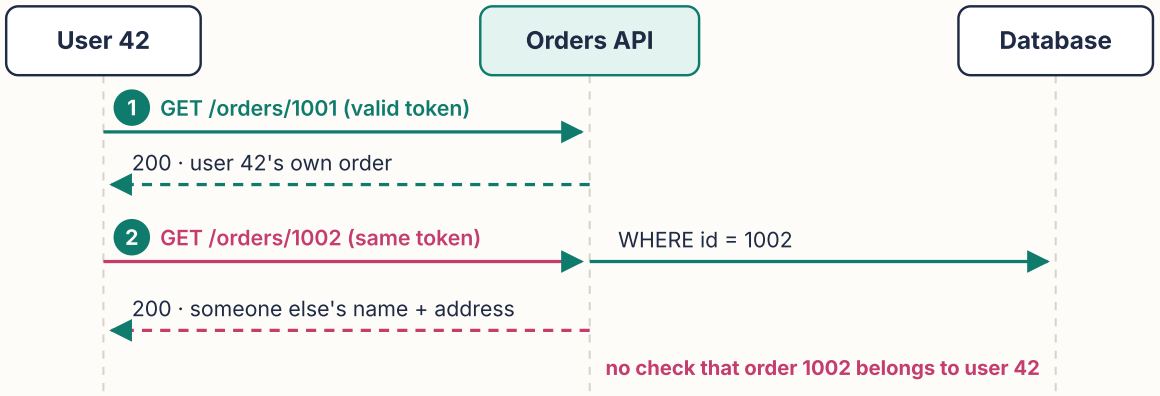


Figure 1. The token is valid both times. The API looks up order 1002 by ID alone and hands it over. This is BOLA, the top API security risk.

2 Scopes: the coarse layer

An OAuth token can carry **scopes** such as `orders:read` or `orders:write`. The gateway can reject a token without `orders:read` before the request even reaches the service.

But scopes describe **kinds of actions**, not **which records**. A token with `orders:read` can still read *any* order, unless the service checks ownership. Scopes narrow what a token can do; they never answer "is this order yours?".

3 BOLA: the missing ownership check

BOLA (Broken Object Level Authorization) is number one in the **OWASP API Security Top 10**. It is also called **IDOR** (Insecure Direct Object Reference). The API takes an object ID from the client and trusts it, without checking that the caller may access that object.

The fix lives in the service, at the data access. Load the order where the ID matches *and* the owner matches the caller's user ID. Take that user ID from the verified token, **never from the request body** or a query parameter the client controls. If nothing comes back, return **404**, so you don't even reveal that order 1002 exists.

In **multi-tenant** systems, also scope every query by tenant ID, ideally automatically: with database **row-level security**, or a repository layer that always adds the filter, so no developer can forget it.

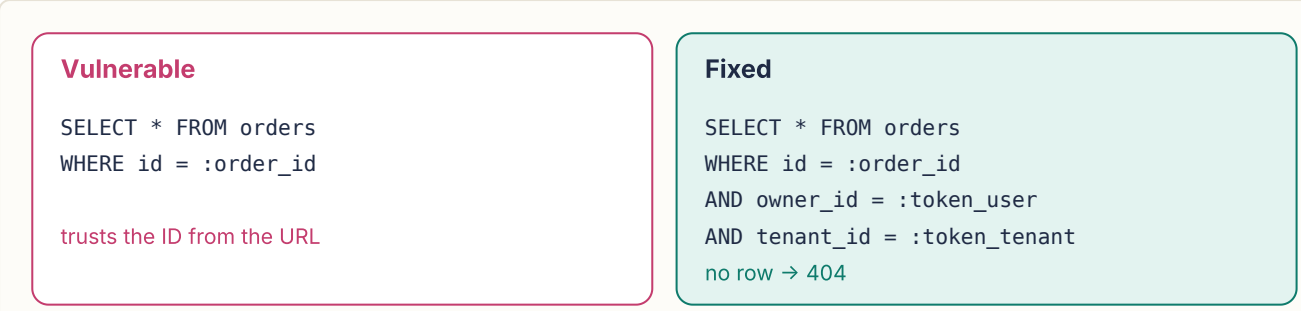


Figure 2. The ownership check belongs in the query. One missing `AND owner_id = …` is the whole vulnerability.

WORKED EXAMPLE: THE ORDER ENDPOINT IN FASTAPI

The route reads `user = Depends(current_user)`, which decodes the token and returns user 42 in tenant 7. The handler runs `SELECT … WHERE id = 1002 AND owner_id = 42 AND tenant_id = 7`. No row comes back, so it returns **404 Not Found**. A support agent with the `support` role takes a different, audited path that checks the role and the tenant, not ownership.

Test it: a test logs in as user A, creates an order, logs in as user B and asserts that `GET`, `PATCH` and `DELETE` on A's order all return 404.

4 Where each check lives

Why not do it all at the gateway? Because **the gateway doesn't know who owns order 1002**. Only the service and its database do. The gateway is great for authentication, token validation and scopes. Object-level checks must live in the service, close to the data.

Internal calls are not exempt. A service calling another service should **pass the user's identity along** (for example the original token, or a signed internal token carrying the user and tenant), instead of skipping checks just because the call came from inside the network.

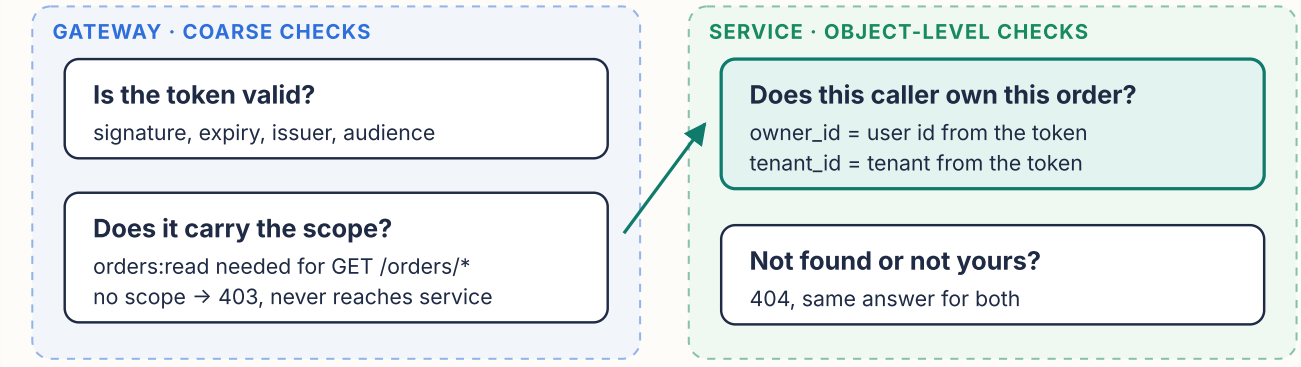


Figure 3. Coarse checks at the edge, fine-grained checks next to the data. A request must pass both.

5 As rules grow, use a model

- **RBAC** (role-based): permissions grouped into roles such as admin or support. Simple, but cannot say “only your own orders” on its own.
- **ABAC** (attribute-based): decide on attributes of the user, resource and context: region, department, owner.
- **ReBAC** (relationship-based), like Google's **Zanzibar**: decide on relationships, such as a document shared with a team the user belongs to.

Centralize the policy, for example in Open Policy Agent, but **enforce it in every service**. Random IDs like UUIDs make guessing harder and are good defense in depth, but they are **never the fix**: IDs leak through links, logs and shared screenshots.

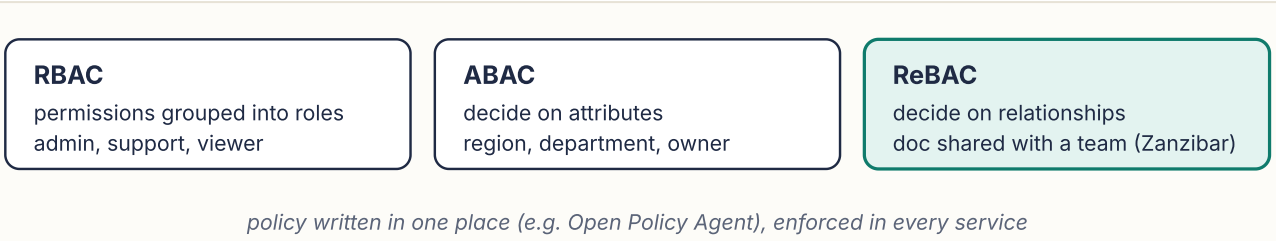


Figure 4. Three ways to express rules. Most real systems combine roles with an ownership or relationship check.

COMMON MISTAKES

- Treating a valid token, or the right scope, as permission to access any record.
- Taking the user or tenant ID from the request body instead of the token.
- Relying on the gateway for object-level checks it cannot make.
- Skipping checks on internal calls, so one compromised service can read everything.
- Switching to UUIDs and calling BOLA fixed.
- Returning 403 for other users' objects, which confirms they exist.

6 Key terms

Authorization: may this caller do this action on this resource

BOLA / IDOR: object ID trusted without an ownership check

Row-level security: database adds the tenant/owner filter itself

Scope: coarse permission on a token, e.g. orders:read

OWASP API Top 10: list of the top API security risks; BOLA is #1

RBAC / ABAC / ReBAC: roles / attributes / relationships

7 Interview questions

Q1. What is BOLA? BASIC

Broken Object Level Authorization: the API accepts an object ID from the client and returns or changes that object without checking the caller may access it, like changing order 1001 to 1002. It is #1 in the OWASP API Security Top 10, also called IDOR.

Q2. What is the difference between authentication and authorization? BASIC

Authentication proves identity (a valid token for user 42). Authorization decides whether that identity may perform this action on this specific resource. A request can pass the first and must still fail the second.

Q3. If a token has the orders:read scope, can it read any order? BASIC

Only if the service lets it. Scopes say what kind of action is allowed, not which records. The service must still check that the order belongs to the caller (and tenant).

Q4. Where should authorization checks live in a gateway + microservices design? SCENARIO

Token validity and scopes at the gateway, rejecting early. Object-level checks in the service at the data layer: every query filtered by owner and tenant from the token, on every request, including internal service-to-service calls.

Q5. You find /invoices/{id} returns any invoice to any logged-in user. How do you fix it and stop it recurring? SCENARIO

Add owner and tenant filters to the query and return 404 when no row matches. Then make it structural: a repository layer or row-level security that always adds the tenant filter, and cross-user tests (user B requests user A's invoice, expect 404) for every endpoint that takes an ID.

Q6. Why return 404 instead of 403 for another user's order? SCENARIO

403 confirms the order exists, which leaks information (an attacker can count orders or probe IDs). 404 gives the same answer for “doesn't exist” and “not yours”.

Q7. We switched order IDs to UUIDs. Is BOLA solved? TRICKY

No. UUIDs only make guessing hard. IDs still leak through URLs, emails, logs and other APIs, and anyone holding one can use it. UUIDs are defense in depth; the ownership check is the fix.

Q8. An internal reporting service calls Orders with a service account that can read everything. What is the risk? TRICKY

It bypasses per-user checks: a bug or compromise in the reporting service exposes every customer. Pass the end user's identity along (original or signed internal token) so Orders still applies owner and tenant rules; give service accounts only the narrow rights they need.

Next: SD-C-017 · API Gateway Explained backendroadmap.com